TALLER DE INTEGRACIÓN III · PROYECTO COFFEEFAST

Guion: Arquitectura de Microservicios CoffeeFast

De dónde a dónde, con qué rutas, puertos y responsabilidades

Autor: **Elías Caranza**

Fecha: **28 de octubre de 2026**

CÓMO USARLO

Leé los recuadros DECIR. Mostrá el archivo y decí el camino: de dónde sale y a dónde llega.

A. Archivos a tener abiertos

> **DECIR**
>
> Tené esto abierto para mostrarlo mientras decís el recorrido:

- Microservices/gateway/nginx.conf
- k8s-next/gateway/nginx.conf, configmap.yaml, deployment.yaml, service.yaml, ingress-api.yaml
- Microservices/docker-compose.yml
- Microservices/ms-cafeterias/src/app.js
- Microservices/ms-menus/src/app.js
- Microservices/ms-pedidos/src/app.js
- Cliente Supabase de la app móvil (Auth y Realtime)

1. Recorrido general

DECIR

Para dibujarlo desde cero: la app móvil o el KDS parte desde ahí y llama al Gateway. El Gateway es Nginx y es el único punto de entrada. Ahí, según la ruta que llega, redirige al microservicio que corresponde. Los tres microservicios van a buscar o guardar lo que necesitan a Supabase Cloud. Ahora bien: el login y ciertos usos de Realtime los hace la app directo a Supabase, sin pasar por el Gateway ni por los microservicios.

2. El Gateway: único punto de entrada

DECIR

El Gateway es el único punto de entrada porque toda petición HTTP hacia los microservicios tiene que pasar por ahí. Ahí se decide el destino. En local, nginx escucha en el puerto 80 del contenedor y se monta desde Microservices/gateway/nginx.conf. En Kubernetes, el Deployment levanta nginx, el ConfigMap guarda ese nginx.conf, el Service lo expone internamente y el Ingress api-cdarwitg.dev.censei.cl apunta al gateway.

MOSTRAR · De dónde sale la config del gateway local

```
# Gateway (nginx) del entorno local de Docker Compose.
#
# Se monta en /etc/nginx/conf.d/default.conf, por eso va SIN el bloque http{}:
# el nginx.conf base de la imagen ya lo incluye dentro de http. El equivalente
# de Kubernetes es k8s-next/gateway/nginx.conf (ese si va completo, porque se
# monta como /etc/nginx/nginx.conf via ConfigMap).
#
# Rutas publicas: /api/cafeterias, /api/menus y /api/pedidos. Todo lo demas
# responde 404. La ruta interna POST /api/menus/precios (ms-pedidos -> ms-menus)
# queda bloqueada a proposito.

server {
  listen 80;
  server_name _;

  location = /gateway/health {
    return 200 'gateway ok\n';
    add_header Content-Type text/plain;
  }

  location ~ ^/api/menus/precios/?$ {
    return 404;
  }

  location /api/cafeterias {
    proxy_pass http://ms-cafeterias:3002;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }

  location /api/menus {
    proxy_pass http://ms-menus:3003;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }

  location /api/pedidos {
    proxy_pass http://ms-pedidos:3005;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }

  location / {
    return 404;
  }
}
```

MOSTRAR · Config del gateway en K8s

```
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /var/run/nginx.pid;

events {
  worker_connections 1024;
}

http {
  include /etc/nginx/mime.types;
  default_type application/octet-stream;

  log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                  '$status $body_bytes_sent "$http_referer" '
                  '"$http_user_agent" "$http_x_forwarded_for"';

  access_log /var/log/nginx/access.log main;
  sendfile on;
  keepalive_timeout 65;

  resolver 10.43.40.162 valid=5s ipv6=off;

  server {
    listen 80;
    server_name _;

    location /gateway/health {
      return 200 'gateway ok\n';
      add_header Content-Type text/plain;
    }

    location ~ ^/api/menus/precios/?$ {
      return 404;
    }

    location /api/cafeterias {
      set $up_caf http://ms-cafeterias.student-cdarwitg.svc.cluster.local:3002;
      proxy_pass $up_caf$request_uri;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;
    }
    location /api/menus {
      set $up_menus http://ms-menus.student-cdarwitg.svc.cluster.local:3003;
      proxy_pass $up_menus$request_uri;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;
    }
    location /api/pedidos {
      set $up_ped http://ms-pedidos.student-cdarwitg.svc.cluster.local:3005;
      proxy_pass $up_ped$request_uri;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
      return 404;
    }
  }
}
```

MOSTRAR · ConfigMap guarda la config

```
apiVersion: v1
data:
  nginx.conf: |-
    worker_processes auto;
    error_log /var/log/nginx/error.log notice;
    pid /var/run/nginx.pid;

    events {
      worker_connections 1024;
    }

    http {
      include /etc/nginx/mime.types;
      default_type application/octet-stream;

      log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                      '$status $body_bytes_sent "$http_referer" '
                      '"$http_user_agent" "$http_x_forwarded_for"';

      access_log /var/log/nginx/access.log main;
      sendfile on;
      keepalive_timeout 65;

      resolver 10.43.40.162 valid=5s ipv6=off;

```

MOSTRAR · Deployment levanta el gateway

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gateway
  namespace: student-cdarwitg
  labels:
    app: gateway
spec:
  replicas: 1
  selector:
    matchLabels:
      app: gateway
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 0
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: gateway
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 101
        fsGroup: 101
      containers:
        - name: gateway
          image: nginx:1.27-alpine
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 80
          volumeMounts:
            - name: config
              mountPath: /etc/nginx/nginx.conf
              subPath: nginx.conf
              readOnly: true
            - name: nginx-cache
              mountPath: /var/cache/nginx
            - name: nginx-run
              mountPath: /var/run
          resources:
            requests:
              cpu: 50m
```

```
                memory: 64Mi
              limits:
                cpu: 150m
                memory: 128Mi
            readinessProbe:
```

MOSTRAR · Service expone el gateway

```
apiVersion: v1
kind: Service
metadata:
  name: gateway
  namespace: student-cdarwitg
  labels:
    app: gateway
spec:
  type: ClusterIP
  ports:
    - name: http
      port: 80
      targetPort: 80
      protocol: TCP
  selector:
    app: gateway

```

MOSTRAR · Ingress: el tráfico llega al gateway

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-cdarwitg-ingress
  namespace: student-cdarwitg
  annotations:
    external-dns.alpha.kubernetes.io/target: proxy.inf.uct.cl
  labels:
    app: gateway
spec:
  ingressClassName: nginx
  rules:
    - host: api-cdarwitg.dev.censei.cl
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: gateway
                port:
                  number: 80

```

3. Rutas: de dónde a dónde

DECIR

Miro las rutas del nginx. Si llega /api/cafeterias, el gateway manda eso a ms-cafeterias. Si llega /api/menus, lo manda a ms-menus. Si llega /api/pedidos, lo manda a ms-pedidos. Todo lo demás lo corta con 404. Y ahí veo /api/menus/precios bloqueado: ese endpoint es solo para que ms-pedidos hable con ms-menus por dentro de la red, la app no lo debe llamar por el gateway.

MOSTRAR · De dónde a dónde: rutas y bloqueo

```
30        proxy_set_header X-Forwarded-Proto $scheme;
31      }
32
33      location /api/menus {
34        proxy_pass http://ms-menus:3003;
35        proxy_set_header Host $host;
36        proxy_set_header X-Real-IP $remote_addr;
37        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
38        proxy_set_header X-Forwarded-Proto $scheme;
39      }
40
41      location /api/pedidos {
42        proxy_pass http://ms-pedidos:3005;
43        proxy_set_header Host $host;
44        proxy_set_header X-Real-IP $remote_addr;
45        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
46        proxy_set_header X-Forwarded-Proto $scheme;
47      }
48
49      location / {
50        return 404;
51      }
52    }
```

DECIR

Puertos para dibujarlos: en docker-compose, el gateway va por GATEWAY_PORT hacia el 80. ms-cafeterias escucha en 3002, ms-menus en 3003, ms-pedidos en 3005. En Kubernetes van a los Services con esos mismos puertos internos.

MOSTRAR · docker-compose: puertos de cada servicio

```
40        image: coffeefaster/ms-cafeterias:local
41        container_name: cf-ms-cafeterias
42        restart: unless-stopped
43        env_file:
44          - .env
45        environment:
46          PORT: 3002
47        ports:
48          - "${MS_CAFETERIAS_PORT:-3002}:3002"
49        extra_hosts:
50          - "host.docker.internal:host-gateway"
51        healthcheck:
52                                                              test:          ["CMD",          "node",          "-e",
      "require('http').get('http://127.0.0.1:3002/health',r=>process.exit(r.statusC...
53          interval: 10s
54          timeout: 3s
55          retries: 5
56          start_period: 5s
57        networks:
58          - cf-ms
59
60      # ------------------------------------------------------------- ms-menus
61      ms-menus:
62        build:
63          context: .
```

```
      dockerfile: ms-menus/Dockerfile
    image: coffeefaster/ms-menus:local
    container_name: cf-ms-menus
    restart: unless-stopped
    env_file:
      - .env
    environment:
      PORT: 3003
    ports:
      - "${MS_MENUS_PORT:-3003}:3003"
    extra_hosts:
      - "host.docker.internal:host-gateway"
    healthcheck:
                                                        test:          ["CMD",          "node",          "-e",
"require('http').get('http://127.0.0.1:3003/health',r=>process.exit(r.statusC...
      interval: 10s
      timeout: 3s
      retries: 5
      start_period: 5s
    networks:
      - cf-ms

  # ------------------------------------------------------------ ms-pedidos
  ms-pedidos:
    build:
      context: .
      dockerfile: ms-pedidos/Dockerfile
    image: coffeefaster/ms-pedidos:local
    container_name: cf-ms-pedidos
    restart: unless-stopped
    env_file:
      - .env
    environment:
      PORT: 3005
      SUPABASE_PUBLISHABLE_KEY: ${SUPABASE_PUBLISHABLE_KEY}
    ports:
      - "${MS_PEDIDOS_PORT:-3005}:3005"
    extra_hosts:
      - "host.docker.internal:host-gateway"
    healthcheck:
                                                        test:          ["CMD",          "node",          "-e",
"require('http').get('http://127.0.0.1:3005/health',r=>process.exit(r.statusC...
      interval: 10s
      timeout: 3s
      retries: 5
      start_period: 5s
    networks:
      - cf-ms

networks:
  cf-ms:
    driver: bridge
```

4. Responsabilidad de cada uno: de dónde a dónde

DECIR · ms-cafeterias

ms-cafeterias recibe las llamadas que llegan por /api/cafeterias desde el gateway y consulta a Supabase lo que corresponda a cafeterías.

MOSTRAR · app.js: solo atiende /api/cafeterias

```
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');

const { HttpError } = require('@coffeefaster/shared');
const cafeteriasRoutes = require('./modules/cafeterias/cafeterias.routes');

const app = express();

app.use(helmet());
app.use(cors());
app.use(morgan('dev'));
app.use(express.json({ limit: '1mb' }));

app.get('/health', (req, res) => {
  res.status(200).json({
    servicio: 'ms-cafeterias',
    version: '1.0.0',
    status: 'healthy',
    timestamp: new Date().toISOString()
  });
});

// UN SOLO RECURSO. Todo lo demas responde 404.
app.use('/api/cafeterias', cafeteriasRoutes);

app.use((req, res) => {
  res.status(404).json({
    error: 'Ruta no encontrada',
    servicio: 'ms-cafeterias',
    recurso: '/api/cafeterias',
    path: req.originalUrl
  });
});

// Manejador de errores: un solo lugar donde se decide el status.
// `err.status` viene de HttpError del modulo compartido; `503` por defecto
// para todo lo demas, porque si llego aqui sin status es infraestructura.
app.use((err, req, res, next) => {
  const status = err instanceof HttpError ? err.status : err.status || 503;

  const cuerpo = {
    error: status >= 500 ? 'Servicio no disponible' : err.message,
    servicio: 'ms-cafeterias'
  };

  // `detalle` es el contexto que el controller adjunto al error (p.ej.
  // "Ids no disponibles: 4242"). HttpError lo guarda en err.detalle. Si el
  // controller no adjunto nada, se omite: repetir err.message solo agrega ruido.
  if (status < 500 && err.detalle) cuerpo.detalle = err.detalle;

  if (status >= 500) {
    console.error(`[ms-cafeterias] ${status} en ${req.method} ${req.originalUrl}:`, err.message);
    if (err.causa) console.error('  causa:', err.causa);
```

DECIR · ms-menus

ms-menus recibe las llamadas por /api/menus desde el gateway y consulta a Supabase el menú y productos. Además, cuando ms-pedidos lo necesita, ms-pedidos le pide los precios por el endpoint interno /api/menus/precios (comunicación entre microservicios, fuera del gateway).

MOSTRAR · app.js: atiende /api/menus

```
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');

const { HttpError } = require('@coffeefaster/shared');
const menusRoutes = require('./modules/menus/menus.routes');

const app = express();

app.use(helmet());
app.use(cors());
app.use(morgan('dev'));
app.use(express.json({ limit: '1mb' }));

app.get('/health', (req, res) => {
  res.status(200).json({
    servicio: 'ms-menus',
    version: '1.0.0',
    status: 'healthy',
    timestamp: new Date().toISOString()
  });
});

// UN SOLO RECURSO. Todo lo demas responde 404.
app.use('/api/menus', menusRoutes);

app.use((req, res) => {
  res.status(404).json({
    error: 'Ruta no encontrada',
    servicio: 'ms-menus',
    recurso: '/api/menus',
    path: req.originalUrl
  });
});

// Manejador de errores: un solo lugar donde se decide el status.
// `err.status` viene de HttpError del modulo compartido; `503` por defecto para
// todo lo demas, porque si llego aqui sin status es infraestructura.
app.use((err, req, res, next) => {
  const status = err instanceof HttpError ? err.status : err.status || 503;

  const cuerpo = {
    error: status >= 500 ? 'Servicio no disponible' : err.message,
    servicio: 'ms-menus'
  };

  // `detalle` es el contexto que el controller adjunto al error (p.ej.
  // "Ids no disponibles: 4242"). HttpError lo guarda en err.detalle. Si el
  // controller no adjunto nada, se omite: repetir err.message solo agrega ruido.
  if (status < 500 && err.detalle) cuerpo.detalle = err.detalle;

  if (status >= 500) {
    console.error(`[ms-menus] ${status} en ${req.method} ${req.originalUrl}:`, err.message);
    if (err.causa) console.error('  causa:', err.causa);
  }

  res.status(status).json(cuerpo);
});

```

DECIR · ms-pedidos

ms-pedidos recibe las llamadas por /api/pedidos desde el gateway. Ahí orquesta: valida que los productos sean de la cafetería, pide los precios a ms-menus (interno), y llama a la RPC procesar_pago para crear el pedido en Supabase. Es decir, ms-pedidos va desde el gateway hacia ms-menus por red interna y hacia la RPC en Supabase.

MOSTRAR · app.js: atiende /api/pedidos

```
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');

const { HttpError } = require('@coffeefaster/shared');
const pedidosRoutes = require('./modules/pedidos/pedidos.routes');

const app = express();

app.use(helmet());
app.use(cors());
app.use(morgan('dev'));
app.use(express.json({ limit: '1mb' }));

app.get('/health', (req, res) => {
  res.status(200).json({
    servicio: 'ms-pedidos',
    version: '1.0.0',
    status: 'healthy',
    timestamp: new Date().toISOString()
  });
});

app.use('/api/pedidos', pedidosRoutes);

app.use((req, res) => {
  res.status(404).json({
    error: 'Ruta no encontrada',
    servicio: 'ms-pedidos',
    recurso: '/api/pedidos',
    path: req.originalUrl
  });
});

app.use((err, req, res, next) => {
  const status = err instanceof HttpError ? err.status : err.status || 503;

  const cuerpo = {
    error: status >= 500 ? 'Servicio no disponible' : err.message,
    servicio: 'ms-pedidos'
  };

  if (status < 500 && err.detalle) cuerpo.detalle = err.detalle;

  if (status >= 500) {
    console.error(`[ms-pedidos] ${status} en ${req.method} ${req.originalUrl}:`, err.message);
    if (err.causa) console.error('  causa:', err.causa);
  }

  res.status(status).json(cuerpo);
});

module.exports = app;

```

5. Lo que va directo a Supabase

DECIR

Auth y ciertos usos de Realtime van directo desde la app móvil a Supabase Cloud, sin pasar por el gateway ni por ningún microservicio. El resto del negocio (cafeterías, menú, pedidos) sí pasa por el gateway hacia los microservicios, y esos microservicios sí van a Supabase con sus credenciales.

6. ms-inventario

DECIR

Dejo esto claro: ms-inventario fue retirado del proyecto. No está en docker-compose, no está en k8s-next y no forma parte de la arquitectura actual.

ACLARACIÓN

ms-inventario NO forma parte de la arquitectura actual.

7. Cierre: trazo completo

DECIR

Para trazarlo completo: App/KDS → Gateway (Nginx, único punto de entrada). Si es /api/cafeterias → ms-cafeterias:3002 → Supabase. Si es /api/menus → ms-menus:3003 → Supabase. Si es /api/pedidos → ms-pedidos:3005 → pide precios a ms-menus internamente y llama a procesar_pago en Supabase. /api/menus/precios bloqueado desde gateway. Auth y algunos Realtime van directo App → Supabase. ms-inventario no lo dibujo porque fue retirado.